



Université Abou Bakr Belkaid
Faculté des sciences
Département d'informatique

COURS GÉNIE LOGICIEL

NIVEAU : L3 INFORMATIQUE/3 EME INGÉNIEUR

Responsable du module : Dr Souad Meziane Tani

E –mail : s.mezianetani13@gmail.com

Septembre 2025

Objectifs du module

- Comprendre comment sont construits les logiciels ;
- Être en mesure d'exprimer des besoins et de suivre un développement de logiciel.
- Vous donner envie de devenir développeur de logiciels !

Organisation du cours

- **Cours**

Une séance 1H30 /semaine.

- **TD**

Une séance 1H30 /semaine

TP: Modélisation UML avec Modelio

Une séance 1H30 /semaine.

Évaluation du module

Note finale = (60% note d'examen, 40% note CC).

Note d'examen : Note de l'examen final .

Note CC = Note CC + Note de TP .

Note de TP : Note de test de TP₊ assiduité .

Contenu de la matière

Chapitre 1: Introduction au génie logiciel

Chapitre 2: Introduction à la modélisation avec UML((Unified Modeling Language)

Chapitre 3: Diagramme UML de cas d'utilisation

Chapitre 4: Diagramme de séquence.

Chapitre 5 : Diagrammes UML de classes et d'objets

Contenu de la matière

Chapitre 6 :Diagramme de séquence objet

Chapitre 7 :Introduction aux méthodes de développement (RUP ,XP).

Chapitre 8:Patrons de conception et leur mise en place au sein du processus de développement .

Chapitre 1 :

Introduction au Génie Logiciel

Plan

1. Définition du génie logiciel .
2. Les enjeux du génie logiciel .
- 3 . Qualité du logiciel .
4. Modèles de cycle de vie d'un logiciel.

Génie Logiciel : c'est quoi ?

✓ **Génie Logiciel** (*software engineering*) (IEEE standards)

Ensemble **des méthodes, des techniques et des outils**
dédiés à **la conception**, au **développement** et à la
maintenance des systèmes informatiques.

Génie logiciel

Idée : appliquer les méthodes classiques **d'ingénierie** au domaine du **logiciel**.

Ingénierie (ou génie) : Ensemble des fonctions allant de la conception et des études à la responsabilité de la construction et au contrôle des équipements d'une installation technique ou industrielle .

Génie , civil, naval, mécanique.

Un logiciel : c'est quoi ?

Un logiciel est l'ensemble des **programmes** et de **documents** nécessaires à leur installation, utilisation, développement et maintenance.

Catégories de logiciel

✓ **Systèmes d'information**

Manipulation de données

Exemples: gestion de stock, réservation de salles

✓ **Systèmes temps réel**

Réaction à des évènements

Exemples: automatisation d'usine, pilotage automatique

✓ **Systèmes distribués**

Coordination de différentes composantes

Exemples: messagerie électronique, Teams

✓ **Systèmes embarqués**

Combinaison de matériel et logiciel spécifiques

Exemples: GAB,

Historique du Génie logiciel

Le GL est apparu à la fin des années 60 pour répondre à la "la crise du logiciel".

Symptôme de cette crise :

- ✓ Le logiciel livré ne satisfait pas les besoins de l'utilisateur,
- ✓ il consomme plus de ressources que prévu et il est à l'origine de pannes.
- ✓ Temps de réponse trop lents.
- ✓ Non respect des délais et des coûts
- ✓ Maintenance difficile, coûteuse et souvent à l'origine de nouvelles erreurs.

Erreurs célèbres . . .

Entre 1985 et 1987, le problème du **Therac-25** (un appareil d'irradiation thérapeutique) : 2 morts, 4 irradiés.



En 1994, Le bug du Pentium. Coût : 500 millions de dollars.

le 4 juin 1996, l'explosion du premier vol d'**Ariane 5**, le bug informatique le plus coûteux de l'histoire.

En 1962, la mission **Mariner 1** a connu un incident similaire

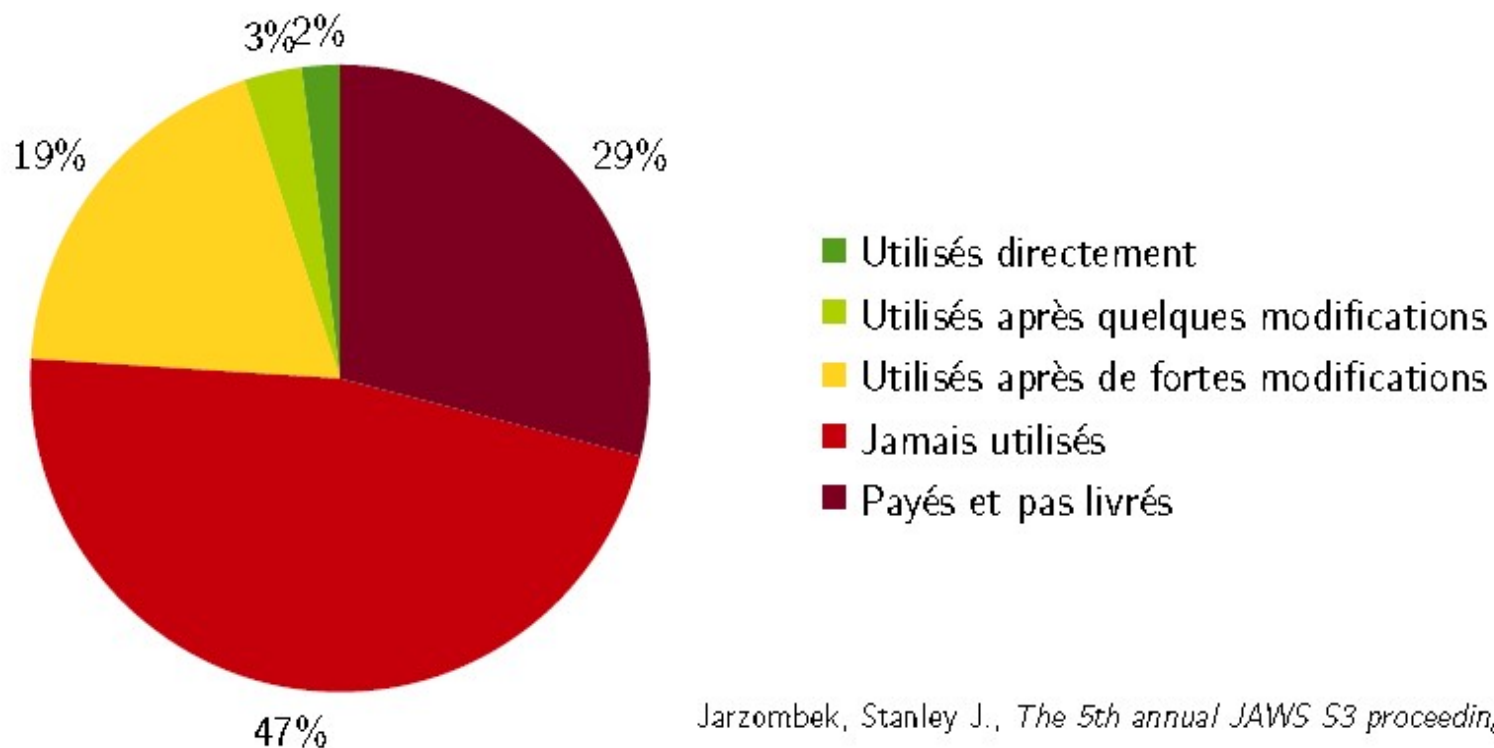
TAURUS, un projet d'informatisation de la bourse londonienne : définitivement abandonné après 4 années de travail et 100 millions de dollars de pertes.

Le bug de l'an 2000 : dysfonctionnements lorsque les dates sont postérieures au 31 décembre 1999.

.....

Étude du DoD 1995

Étude du *Department of Defense des États-Unis* sur les logiciels produits dans le cadre de 9 gros projets militaires



Jarzombek, Stanley J., *The 5th annual JAWS S3 proceedings*, 1999

Génie Logiciel : Difficultés

➤ Difficultés liées à la nature du logiciel

un logiciel ne s'use pas, sa fiabilité ne dépend que de sa conception
pour rester utilisé un logiciel doit évoluer (changements fréquents)

➤ Les difficultés liées aux personnes

ne savent pas toujours ce qu'elles veulent, ou ne savent pas bien l'exprimer
communication difficile entre personnes de métiers différents (jargons) beaucoup
d'autodidactes qui croient savoir,

➤ Les difficultés technologiques

courte durée de vie du matériel,
beaucoup de méthodes et de langages
évolution des outils de développement,
...

LES PRINCIPES DU GENIE LOGICIEL (Carlo Ghezzi)

1. La rigueur :

- Précision / exactitude
- Modèles mathématiques

2. La Généralisation

- généricité et héritage
- Réutilisabilité

3. La Structuration

- Modele << bottom-up >>
- Modele << top-down >>

4. La modularité :

5. L'abstraction :

- Modelisation d'un fait
- Vue partielle

6. La construction incrémentale

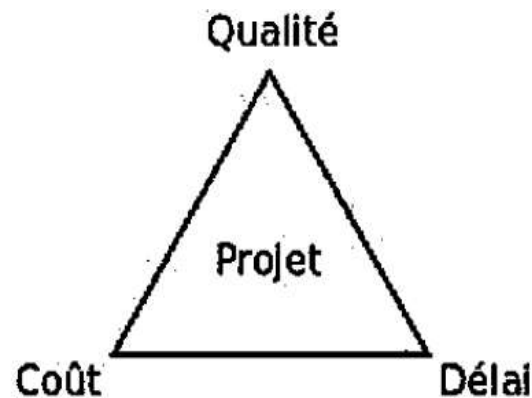
- Evolutivite
- Reutilisabilite
- maintenabilité

7 . La Documentation

8. La Vérification

Enjeu du génie logiciel ?

Bien développer le bon logiciel.



Comment faire **des logiciels de qualité** ?

Qu'attend-on d'un logiciel ? Quels sont **les critères de qualité** ?

Qualité logiciel

Critères de qualité de (Bertrand Meyer)

- ❑ **Validité** : réponse aux besoins des utilisateurs
- ❑ **Facilité d'utilisation(ergonomie)** : prise en main et robustesse
- ❑ **Performance** : temps de réponse, débit, fluidité...
- ❑ **Fiabilité** : tolérance aux pannes
- ❑ **Sécurité** : intégrité des données et protection des accès
- ❑ **Maintenabilité** : facilité à corriger ou transformer le logiciel
- ❑ **Portabilité** : changement d'environnement matériel ou logiciel

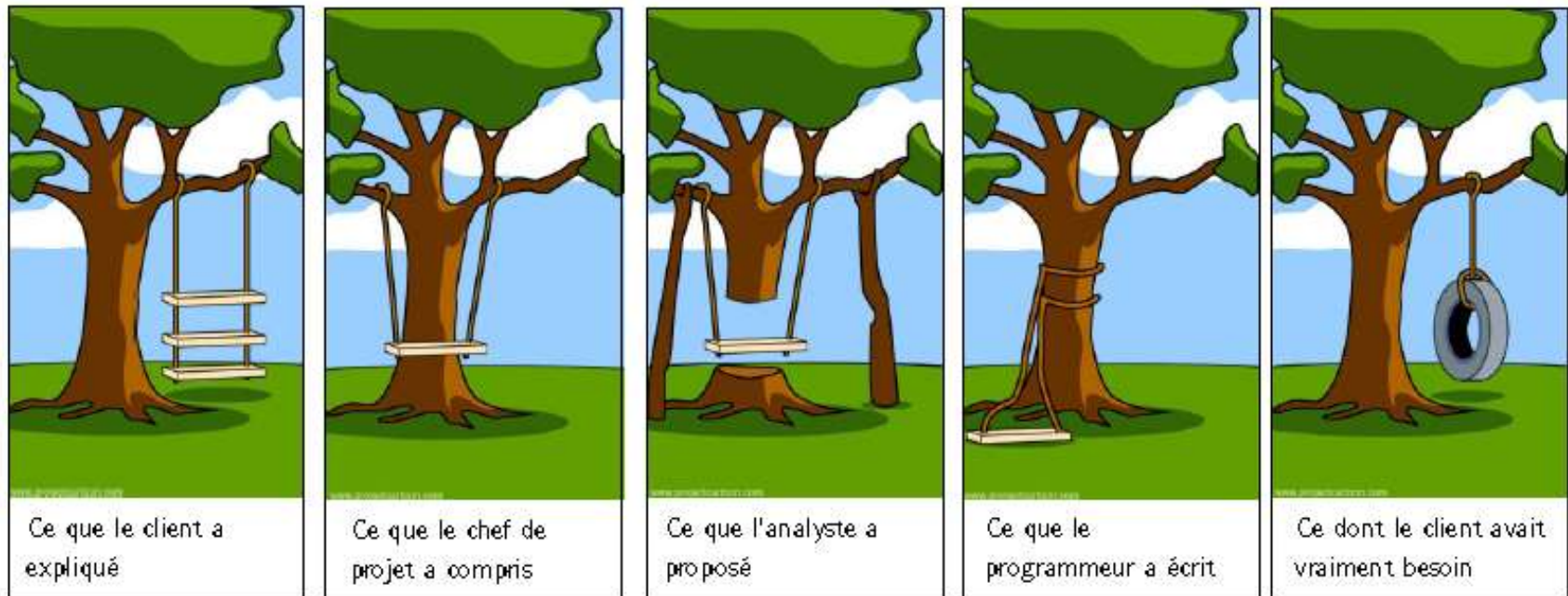
Critère de qualité

La validité (correction, justesse, conformité) est la capacité que possède un produit logiciel à remplir exactement ses fonctions, définies par le cahier des charges et les spécifications.

Adéquation entre :

- **Le besoin effectif de l'utilisateur**
- **Les fonctions offertes par le logiciel**

Critère de qualité(validité)



Solutions :

- Emphase sur l'analyse des besoins
- Améliorer la communication (langage commun, démarche participative)
- Travailler avec rigueur

Critère de qualité

Fiabilité(robustesse) est la capacité qu'offrent des systèmes logiciels à réagir de manière appropriée à la présence de conditions anormales (i.e. rien de catastrophique ne peut survenir, même en dehors des conditions d'utilisation prévues).

Solutions :

- Utiliser des méthodes formelles, des langages et des méthodes de programmation de haut niveau
- Vérifications, tests

Critère de qualité

Facilité d'utilisation(Ergonomie)

La facilité d'utilisation est la facilité avec laquelle des personnes présentant des formations et des compétences différentes peuvent apprendre à utiliser les produits logiciels et s'en servir pour résoudre des problèmes.

- **Facilité d'apprentissage** : comprendre ce que l'on peut faire avec le logiciel, et savoir comment le faire
- **Facilité d'utilisation** : importance de l'effort nécessaire pour utiliser le logiciel à des fins données

Solutions :

- Analyse du mode opératoire des utilisateurs
- Adapter l'ergonomie des logiciels aux utilisateurs.

Critère de qualité

Compatibilité (Interopérabilité)

La compatibilité est la facilité avec laquelle des éléments logiciels peuvent être combinés à d'autres.

Un logiciel doit pouvoir interagir en synergie avec d'autres logiciels

Solutions :

- Bases de données (découplage données/traitements)
- « Externaliser » certaines fonctions en utilisant des « Middleware » avec une API (Application Program Interface) bien définie
- Standardisation des formats de fichiers (XML...) et des protocoles de communication (CORBA...)
- Les ERP (Enterprise Resources Planning)

Critère de qualité

Efficacité (performance)

L'efficacité est la capacité d'un système logiciel à utiliser le minimum de ressources matérielles, que ce soit le temps machine, l'espace occupé en mémoire externe et interne, ou la bande passante des moyens de communication.

Les logiciels doivent satisfaire aux contraintes de temps d'exécution

Solutions :

Logiciels plus simples

Veiller à la complexité des algorithmes

Machines plus performantes

Critère de qualité

Portabilité

La portabilité est la facilité avec laquelle des produits logiciels peuvent être transférés d'un environnement logiciel ou matériel à l'autre.

Un même logiciel doit pouvoir fonctionner sur plusieurs machines



Solutions :

Rendre le logiciel indépendant de son environnement d'exécution (voir interopérabilité)

Machines virtuelles

Critère de qualité

Réutilisabilité

La réutilisabilité est la capacité des éléments logiciels à servir à la construction de plusieurs applications différentes.

On peut espérer des gains considérables car dans la plupart des logiciels :

- ❖ 80 % du code est du « tout venant » qu'on retrouve à peu près partout
- ❖ 20 % du code est spécifique

Solutions :

Abstraction, généricité

Construire un logiciel à partir de composants prêts à l'emploi

Design Patterns

Critère de qualité

Maintenabilité

La maintenabilité est le degré de facilité de la maintenance d'un produit logiciel.

Un logiciel ne s'use pas

Pourtant, la maintenance absorbe une très grosse partie des efforts de développement (représente 67 % de l'effort de développement) ;

Les coûts de maintenance se jouent très tôt dans le processus d'élaboration du logiciel

Solution :

Réutilisabilité, modularité

Vérifier, tester

Anticiper les changements à venir.



Critère de qualité

Extensibilité

L'extensibilité est la facilité d'adaptation des produits logiciels aux changements de spécifications.

Intégrité

Aptitude d'un logiciel à protéger son code et ses données contre des accès non autorisés.

Ponctualité

La ponctualité est la capacité d'un système logiciel à être livré au moment désiré par ses utilisateurs, ou avant.

C'est quoi un bon logiciel bien fait ?

Un bon logiciel bien fait est un logiciel

- **correct (valide),**
- **fiable (robuste),**
- **avec un code réutilisable**
- **compatible avec d'autres logiciels,**
- **efficace,**
- **portable,**
- **facile à utiliser,**
- **maintenable,**
- **ponctuel et extensible**

Pourquoi c'est si important?

Aujourd'hui, le logiciel contrôle le monde!

- Processus métiers (administrative etc.)
- Gouvernement
- L'industrie (Usines, chaines de fabrication)
- Transports
- Défense, finance, santé...
- Edition, médias...
- Les nouvelles technologies du web, commerce électronique...

Et bien plus!! Des enjeux aussi bien économiques que politique

Processus de développement logiciel

Processus de développement logiciel : c'est quoi ?

Ensemble **d'activités successives**, organisées en vue de la production d'un logiciel

Activité

Les activités d'un processus sont souvent décrites en termes de :

- ❑ **Entrées de l'activité** (matière première)
- ❑ **Sorties de l'activité** (résultat)
- ❑ **Intervenants** et rôles (qui ?)
- ❑ **Description de l'activité** (quoi - quel est le problème à traiter ?) Standards, guides, « best practices » à appliquer (comment ?)



Activités du développement logiciel

- **Etude de faisabilité.**
- **Analyse des besoins.**
- **Spécification.**
- **Conception.**
- **Programmation.**
- **Validation et vérification.**
- **Livraison.**
- **Maintenance.**

Faisabilité (pourquoi ?)

➤ **Etude préalable ou étude de faisabilité** : Elle concerne la définition globale du problème.

Questions

Pourquoi développer le logiciel ?

Y a-t-il de meilleures alternatives ?

Comment procéder pour faire ce développement ?

Y a-t-il un marché pour le logiciel ?

Quels moyens faut-il mettre en oeuvre ? A-t-on le budget, le personnel,

le matériel nécessaires ?

Spécification (quoi ?)

□ Analyse des besoins :

Comprendre les besoins du client

- Objectifs généraux, environnement du futur système, ressources disponibles, contraintes de performance...
- Fournie par le client (expert du domaine d'application, futur utilisateur...)

□ Spécification :

- Établir une description claire de ce que doit faire le logiciel (fonctionnalités détaillées, exigences de qualité, interface...)
- Clarifier le cahier des charges (ambiguïtés, contradictions) en listant **les exigences fonctionnelles et non fonctionnelles**

Exigences fonctionnelles et non fonctionnelles

Exigences fonctionnelles

Décrivent les fonctionnalités du système ,ce qu'un système doit faire .

Exemple : Distributeur GAB

-Retirer Argent , consulter solde ,Verser argentect

Exigences non fonctionnelles

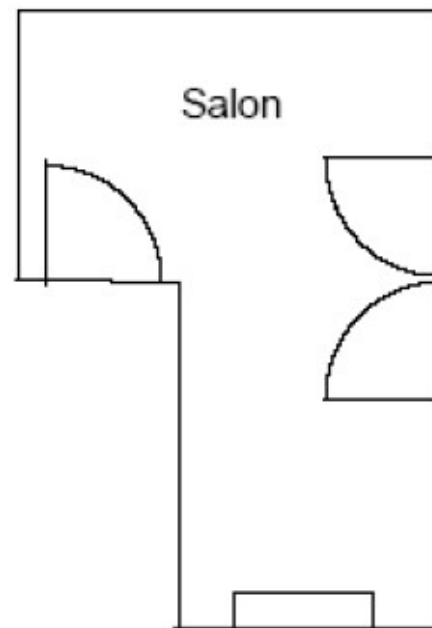
✓ Définissent les propriétés et contraintes du système **Exemple :** Fiabilité,Temps de réponse, Taille de stockage

✓ Exigences sur le processus de développement

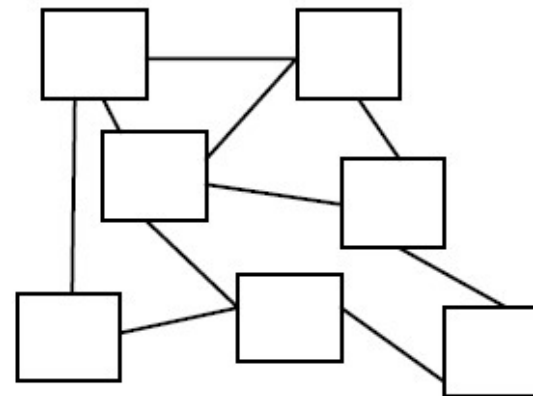
Exemple :Environnement de développement , Langage de programmation , Méthode de développement

Conception (comment ?)

- ❑ **Conception** : Élaborer une solution concrète réalisant la spécification
- **Conception globale** : description architecturale en composants (avec interface et fonctionnalités)



plan général



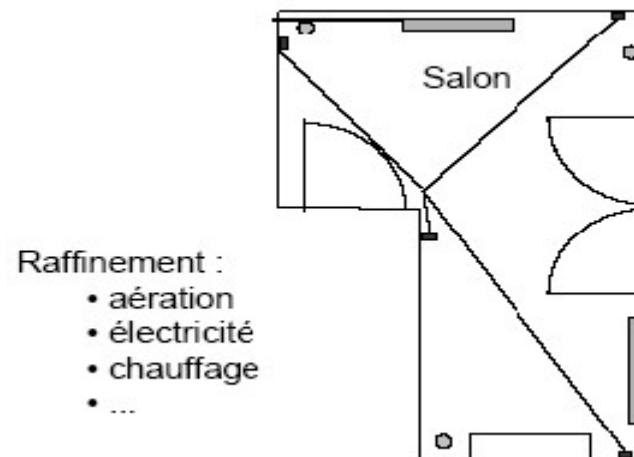
Conception (comment ?)

- **Conception détaillé**

Détailler le fonctionnement des composants

Définir quelques algorithmes, la représentation des données,
...

des tests unitaires sont définis pour s'assurer que les
composants réalisés sont bien conformes à leurs descriptions



Programmation

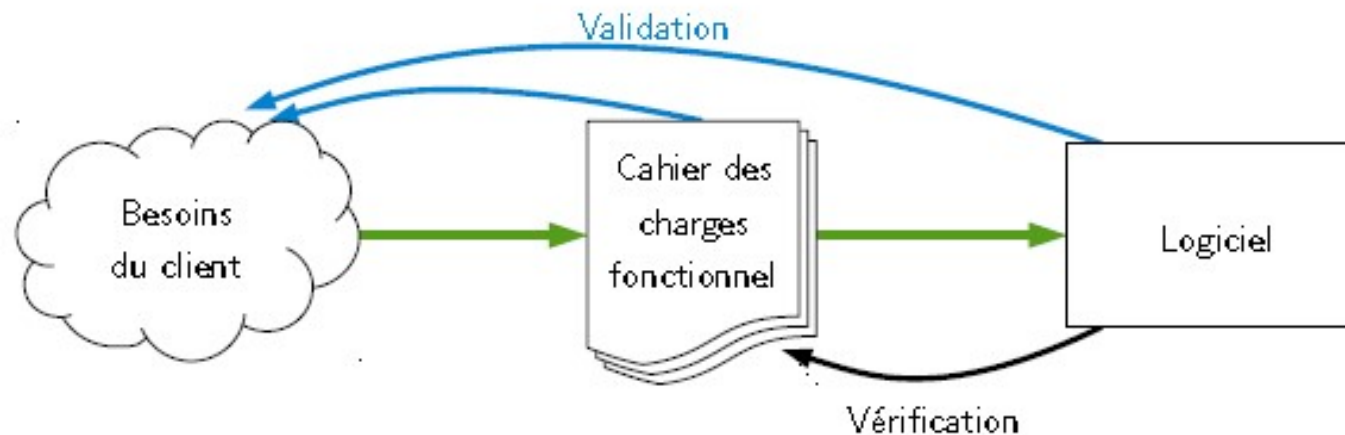
□ **Programmation** : Implantation de la solution conçue

- Choix de l'environnement de développement, du/des langage(s) de programmation, de normes de développement...



Validation et vérification

- ❑ **Validation** : assurer que les besoins du client sont satisfaits (au niveau de la spécification, du produit fini...)
- ❑ **Vérification** : assurer que le logiciel satisfait sa spécification



Maintenance

Types de maintenance :

-
- **Corrective** : identifier et corriger des erreurs trouvées après la livraison
 - **Adaptative** : adapter le logiciel aux changements dans l'environnement (format des données, environnement d'exécution...)
 - **Perfective** : améliorer la performance, ajouter des fonctionnalités, améliorer la maintenabilité du logiciel.

Modèles de cycle de vie d'un logiciel

Modèles de cycle de vie d'un logiciel

Linéaires

- Cascade
- Modèle en « V »

Non-linéaires

- Prototypage
- Modèles incrémentaux
- Modèle en spirale

Cycle de vie en cascade(Waterfall model)

Le modèle en cascade (Waterfall model) est le plus classique des cycles de vie.

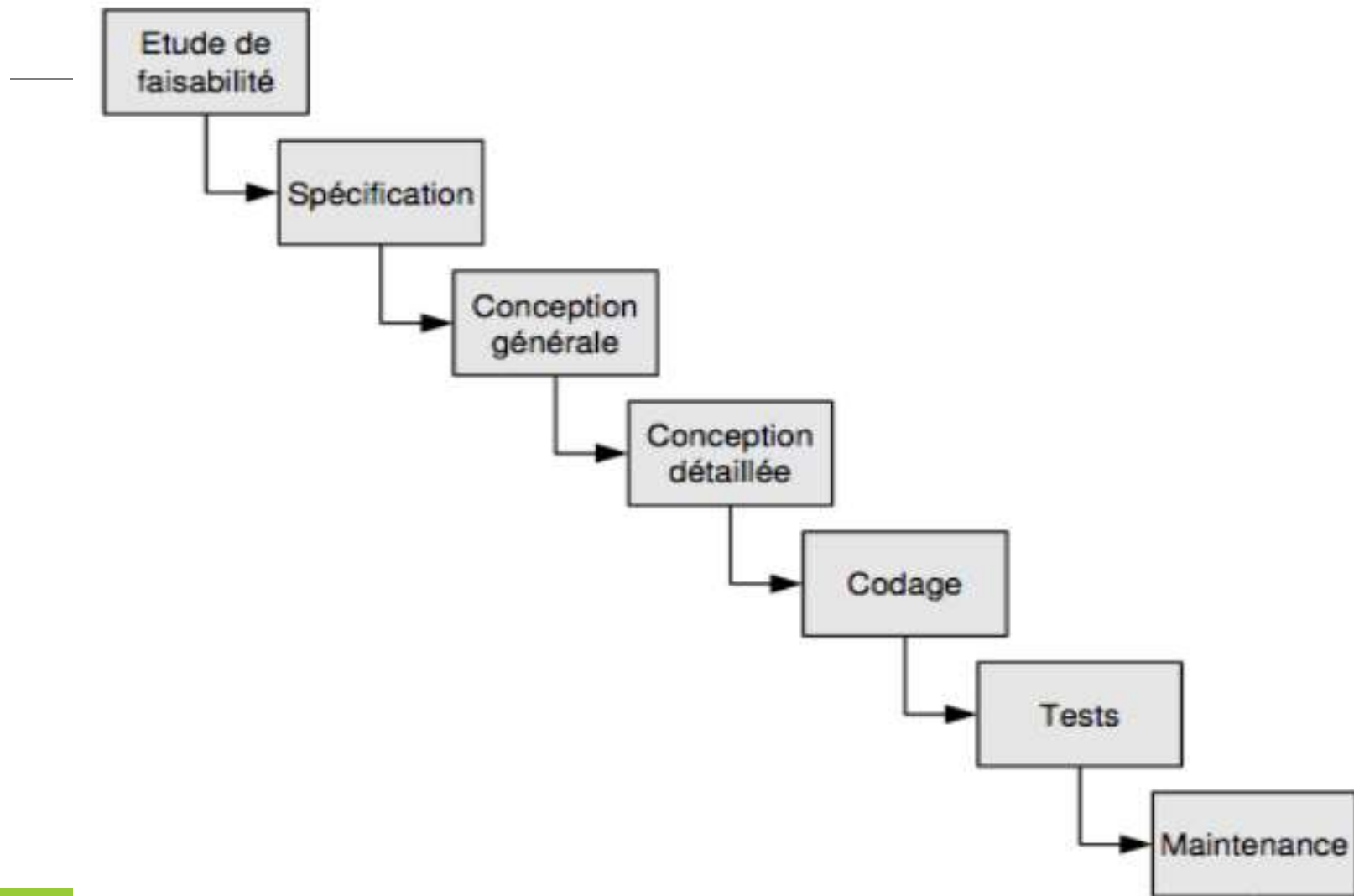
Cycle de vie linéaire sans aucune évaluation entre le début du projet et la validation

Le projet est découpé en phases successives dans le temps

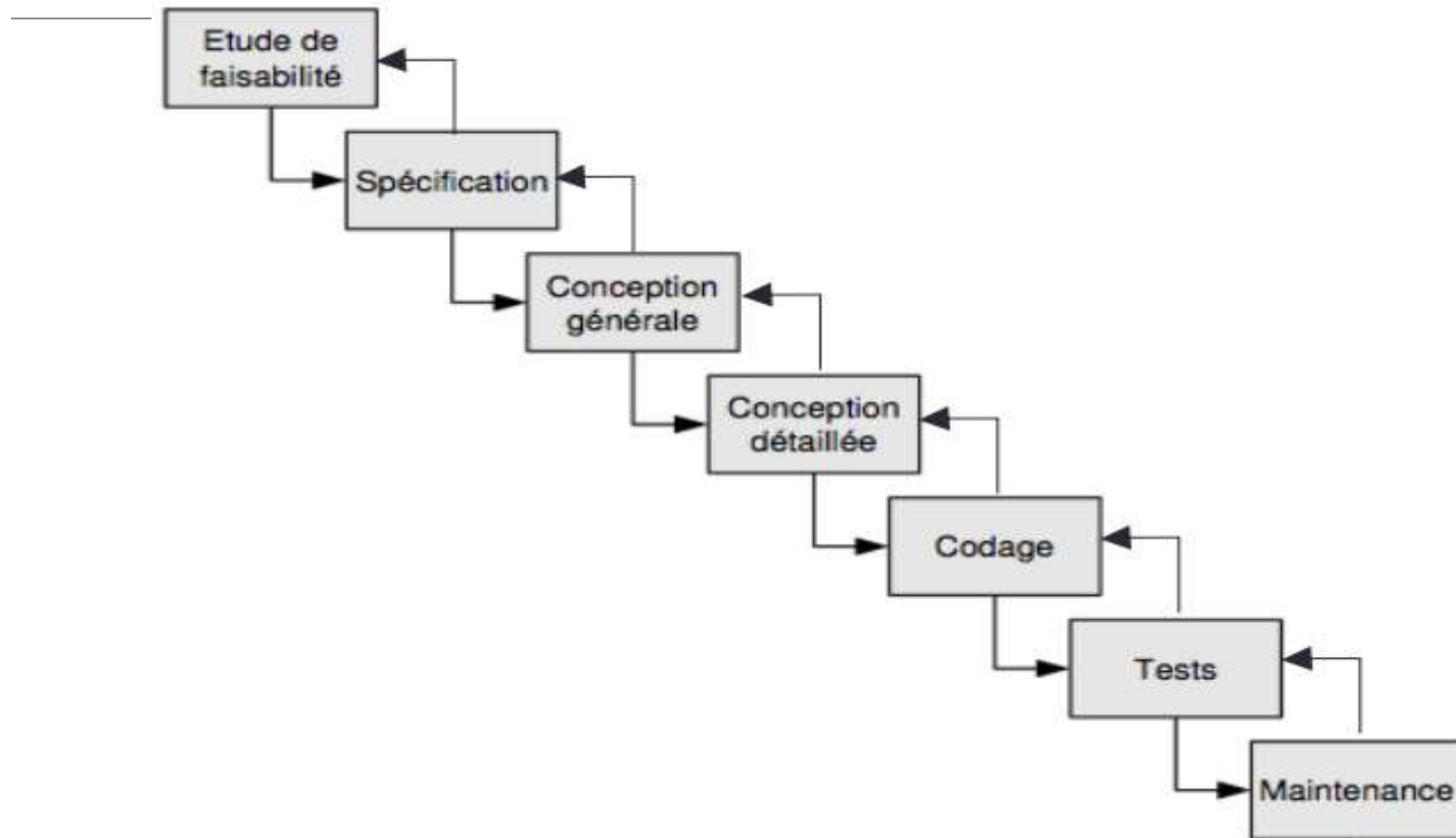
A chaque phase correspond une activité principale bien précise produisant un certain nombre de livrables

Chaque phase ne peut remettre en cause que la phase précédente

Cycle de vie en cascade(Waterfall model)



Cycle de vie en cascade avec retour en arrière



Cycle de vie en cascade

Propriétés

1. Séquentielle
2. Phases
3. Documents

Limites

- Les vrais projets suivent rarement un développement séquentiel
- Établir tous les besoins au début d'un projet est difficile

Sensibilité à l'arrivée de nouvelles exigences : refaire toutes les étapes

+ Bien adapté lorsque les besoins sont clairement identifiés et stables

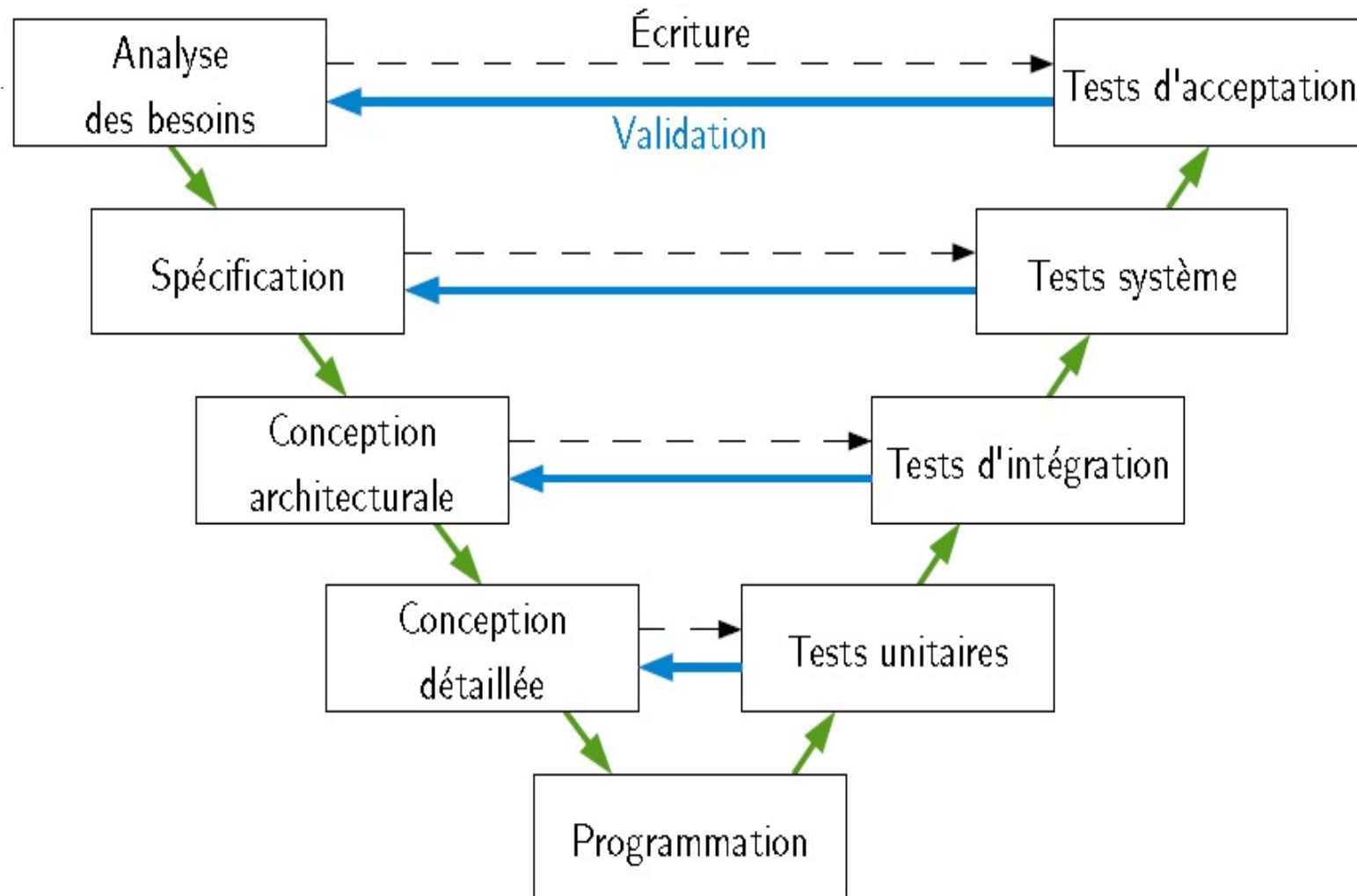
Cycle de vie en « V »

A ce jour, le cycle en V reste le cycle de vie le plus utilisé.
C'est un cycle de vie orienté test :

A chaque activité créative (spécification, conception et codage) correspond une activité de vérification (validation, intégration, tests unitaires).

Chaque phase en amont prépare la phase correspondante de vérification (la vérification est prise en compte au moment même de la création).

Cycle de vie en « V »



Cycle de vie en « V »

Avantages

- La préparation des dernières phases (validation-vérification) par les premières (construction du logiciel), permet d'éviter d'énoncer une propriété qu'il est impossible de vérifier objectivement après la réalisation.

Inconvénients

Construit-on le bon logiciel ? Le logiciel est **utilisé très (trop) tard**.

- Il faut attendre longtemps pour savoir si on a construit le bon logiciel.
- Difficile d'impliquer les utilisateurs lorsque qu'un logiciel utilisable n'est

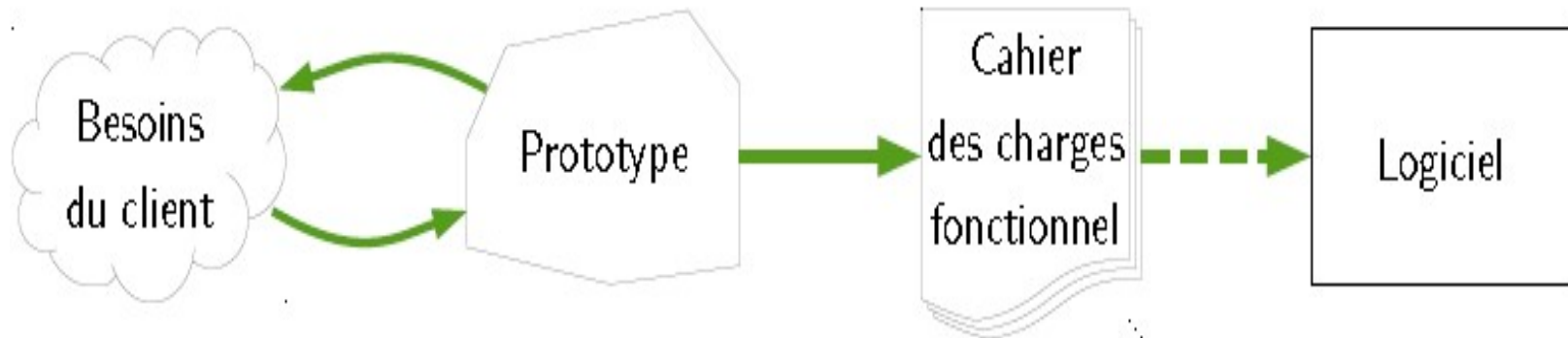
disponible qu'à la dernière phase

Idéal quand les besoins sont bien connus, quand l'analyse et la conception sont claires

Prototypage

Principe :

- Développement rapide d'un prototype avec le client pour valider ses besoins
- Écriture de la spécification à partir du prototype, puis processus de développement linéaire



Avantage : Validation concrète des besoins, moins de risques d'erreur de spécification

Prototypage

- **Le prototypage Jetable** : ici, le squelette du logiciel n'est créé que dans un but et dans une phase particulière du développement.
- **Le prototypage Evolutif** : ici, on conserve tout, au long du cycle de développement. Il est amélioré et complété pour obtenir le logiciel final

Prototypage

Avantage :

- Les efforts consacrés au développement d'un prototype sont le plus souvent compensés par ceux gagnés à ne pas développer de fonctions inutiles

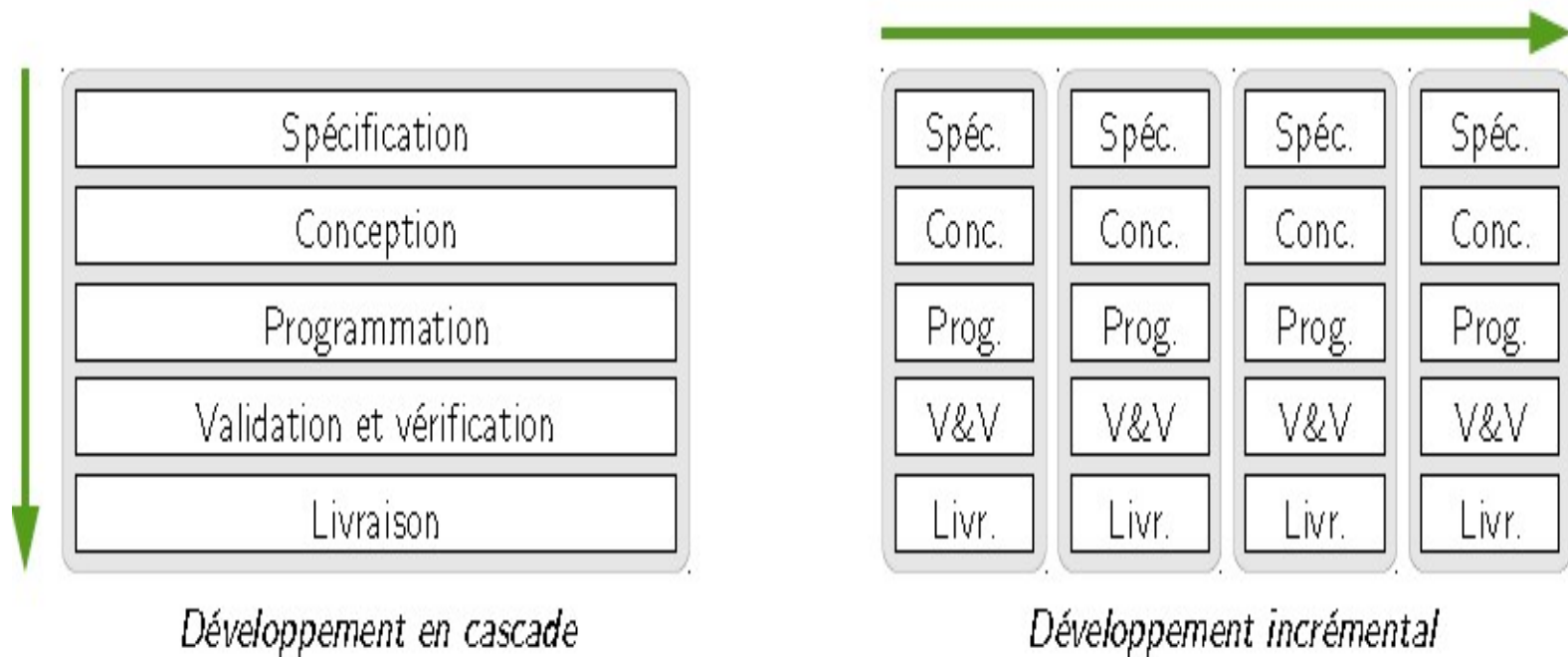
Mais

- ✓ Les décisions rapides sont rarement de bonnes décisions
- ✓ Le prototype évolutif donne-t-il le produit demandé ?

Modèle incrémental

Principe : Développement pas à pas

1. Sous-ensemble minimal et fonctionnel du système
2. Ajouts d'incrément jusqu'à la fin du processus



Modèle incrémental

Avantage

- chaque développement est moins complexe;
- les intégrations sont progressives ;
- possibilité de livraisons et de mises en service après chaque incrément ;

Limite

- la remise en cause du noyau de départ ;
- la remise en cause des incréments précédents;
- l'impossibilité d'intégrer un nouvel incrément

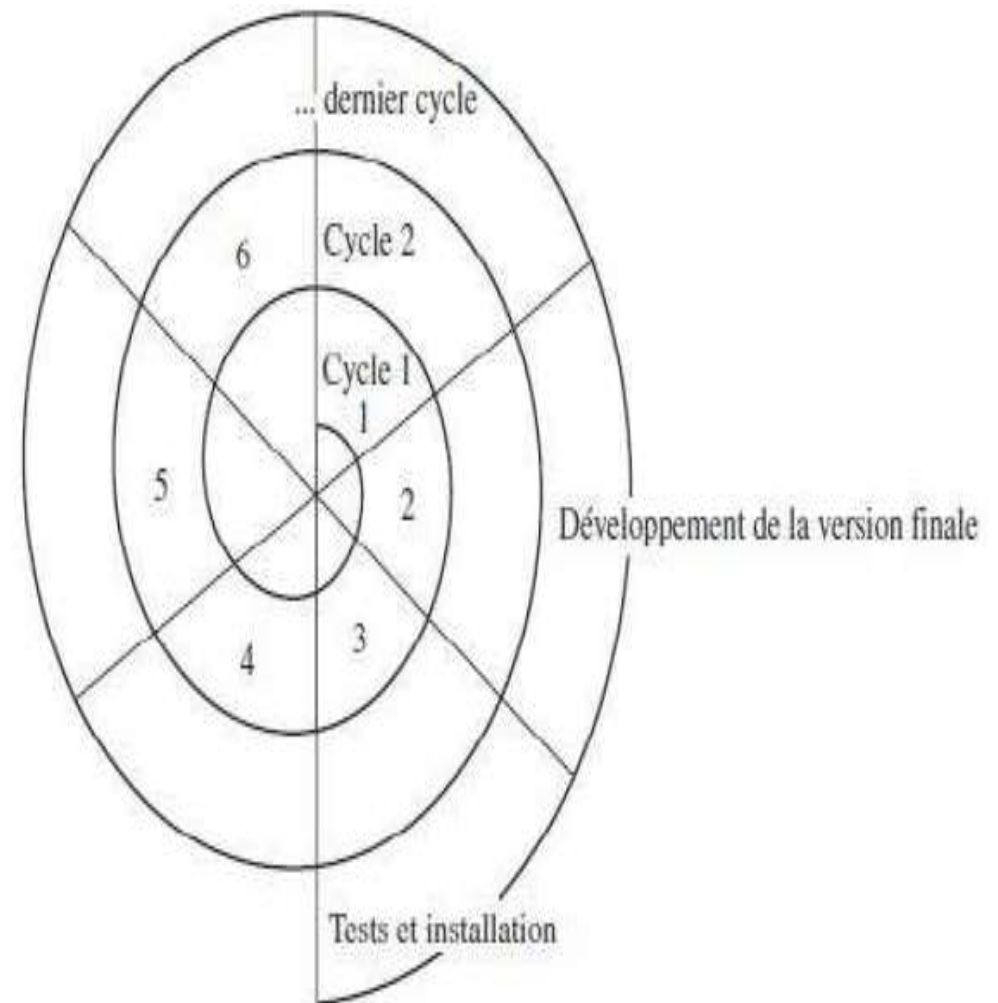
Modèle en Spiral

Plusieurs cycles

1. Consultation du client
2. Analyse des risques
3. Conception
4. Implémentation
5. Tests
6. Planification du prochain cycle

Avantage

- Meilleure maîtrise des risques



Processus de développement logiciel

En pratique :

- Pas de processus idéal
- Choix du processus en fonction des contraintes (taille des équipes, temps, qualité...)
- Adaptation de « processus types » aux besoins réels

Documentation

Objectif : Traçabilité du projet

Pour l'équipe

- Regrouper et structurer les décisions prises
- Faire référence pour les décisions futures
- Garantir la cohérence entre les modèles et le produit

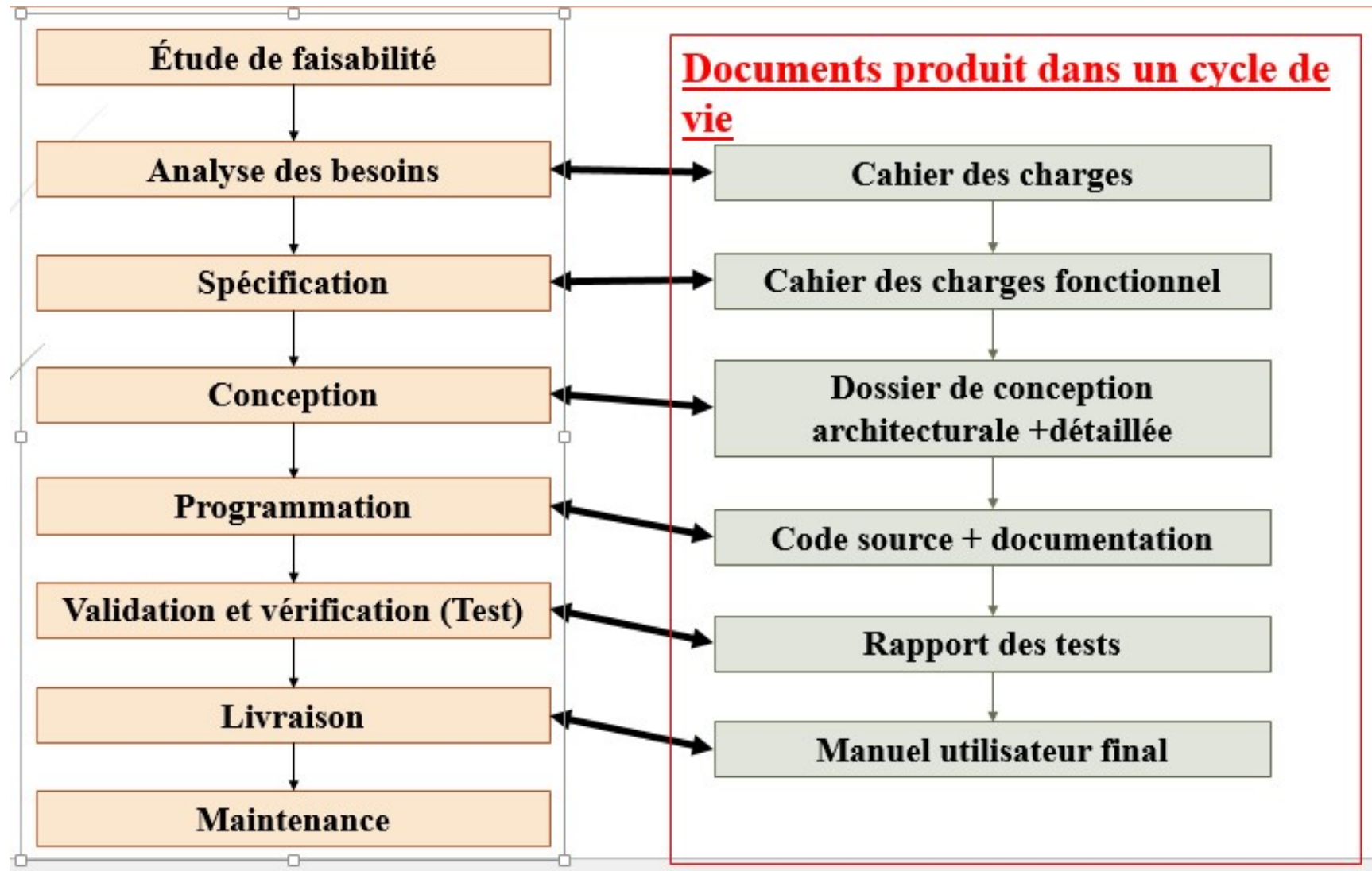
Pour le client :

- Donner une vision claire de l'état d'avancement du projet

Base commune de référence :

- ✓ Personne quittant le projet : pas de perte d'informations
- ✓ Personne rejoignant le projet : intégration rapide

Documents produits dans un cycle de vie



Qu'est-ce qu'un cahier des charges logiciel

- Un document recensant les spécifications/exigences d'un produit logiciel. Il formule les **fonctions** que le système devra accomplir pour le client.
- Il sert de **guide** et de **référence** tout au long du processus de développement.
- S'il est bien conçu, il **garantit** la cohérence et la qualité du produit final

Qu'est-ce qu'un cahier des charges logiciel

- Il facilite la **communication** entre le client et les développeurs tout en permettant d'éviter les problèmes **d'interprétation**.
- Il définit les **responsabilités** et les **rôles** respectifs des parties engagées dans le projet.
- Détermine les modalités de règlement en cas de **litiges**.
- Un élément de base pour la signature d'un **contrat**

Éléments clés d 'un cahier des charges logiciel

un cahier de charge complet et précis comprend:

- Énoncé du **problème** à résoudre.
- Une description du **contexte** d'utilisation du logiciel (son environnement) et des enjeux associés au projet.
- Les **objectifs**, les **fonctionnalités** attendues (spécification fonctionnelle définissant toutes les fonctions offertes par le logiciel) et les **scénarios** d'utilisation.
- Les **contraintes** de réalisation, techniques et opérationnelles, telles que le langage de programmation et les normes à suivre

Éléments clés d'un cahier des charges logiciel

- Les critères de **validation** (comment mesurer la conformité par rapport aux attentes du client).
- **Performances** requises (temps de réponse, sécurité,...).
- Comportement en cas d'**erreurs**.
- Le **planning** et les **ressources** (humaines et matérielles).
- **Interface** avec l'utilisateur et avec d'autres logiciels.
- **Priorités** éventuelles.

Structuration du cahier des charges logiciel

- 1.Introduction** : objectifs du projet et contexte général.
- 2.Exigences fonctionnelles** : description des fonctionnalités attendues.
- 3.Exigences non fonctionnelles** : performance, sécurité, compatibilité, etc.
- 4.Contraintes techniques** : technologies, frameworks, plateformes.
- 5.Critères de validation** : tests, indicateurs de succès.
- 6.Planning** : étapes du projet et délais.
- 7.Annexes** : documents supplémentaires

Structuration du cahier des charges selon la norme **IEEE/ANSI 830-1998**

1. Introduction

- (a) Purpose of the requirements document
- (b) Scope of the product
- (c) Definitions, acronyms and abbreviations
- (d) References
- (e) Overview of the reminder of the document

2. General description

- (a) Product perspective
- (b) Product functions
- (c) User characteristics
- (d) General constraints
- (e) Assumptions and dependencies

3. Specific requirements

Cover functional, non-functional and interface requirements. This is obviously the most substantial part of the document but because of the wide variability in organisational practice, it is not appropriate to define a standard structure for this section. The requirements may document external interfaces, describe system functionality and performance, specify logical database requirements, design constraints, emergent system properties and quality characteristics.

4. Appendices

5. Index

Documents de spécification et conception

Rédaction : le plus souvent en langage naturel (français)

Problèmes :

- Ambiguïtés **plusieurs** sens d'un même mot selon les personnes ou les contextes.
- Contradictions, oublis, redondances difficiles à détecter
- Difficultés à trouver une information.
- Mélange entre les niveaux d'abstraction (spécification vs. Conception)

Documents de spécification et conception

Alternatives au langage naturel

Langages informels :

- **Langage naturel structuré** : modèles de document et règles de rédaction précis et documentés
- **Pseudo-code** : description algorithmique de l'exécution d'une tâche, donnant une vision opérationnelle du système

Langages semi-formels :

- **Notation graphique** : diagrammes accompagnés de texte structuré, donnant une vue statique ou dynamique du système

Langages formels :

- **Formalisme mathématique** : propriétés logiques ou modèle du comportement du système dans un langage mathématique

Documents de spécification et conception

Langages informels ou semi-formels :

✓ Avantages : intuitifs, fondés sur l'expérience, facile à apprendre et à utiliser, répandus

✗ Inconvénients : ambigus, pas d'analyse systématique

Langages formels :

✓ Avantages : précis, analysables automatiquement, utilisables pour automatiser la vérification et le test du logiciel

✗ Inconvénients : apprentissage et maîtrise difficiles

En pratique : utilisation de **langages formels** principalement pour logiciels critiques, ou restreinte aux parties critiques du système

Modélisation

Modèle : Simplification de la réalité, abstraction, vue subjective . modèle météorologique, économique, démographique...

Modéliser un concept ou un objet pour :

- Mieux le comprendre (modélisation en physique)
- Mieux le construire (modélisation en ingénierie)

En génie logiciel :

- Modélisation = spécification + conception
- Aider la réalisation d'un logiciel à partir des besoins du client

Modélisation graphique

Principe : « *Un beau dessin vaut mieux qu'un long discours* »

Seulement s'il est compris par tous de la même manière

